

Technical Meeting

by Tech Lead Szymon



Agenda

- Git guidelines
- Code quality tooling
- Architecture concepts
- Testing guidelines
- Choosing the technology
- First task overview (UMS)

```
resp_iter = self.stub.GetDataResponse()
statuses = {}
async for data in resp_iter:
    status = Status(
        status_id=data.id, name=data.name
    )
    statuses[status.name] = status
return statuses
```

Git guidelines - Github Flow

A lightweight, branch-based development workflow designed for continuous deployment

Always update (fetch) master branch before.

- ``git checkout master``
- ``git pull``

Create a branch for your work:

- For features: ``git checkout -b feat/<name>``
- For bug fixes: ``git checkout -b fix/<name>``

Register your branch in the online repo

- ``git push -u origin <branch-name>``

Working...

To stay in sync with the team while on the feature branch:

- ``git fetch origin``
- ``git merge master`` or ``git rebase master``
- ``git push``

Submit your changes for review:

- ``<repo_url>/pull/new/<branch-name>``
- Squash merge the feature branch into master.

Git guidelines - Github Flow - Why?

- Isolation:

Your branch is your development space.

Unfinished code is never deployed to production.

- Collaboration & Quality Control:

Pull Requests create a dedicated space for peer code reviews, discussion.

- Conflict Prevention:

Regularly syncing with ``master`` resolves integration issues in small chunks rather than massive merge conflicts.

- Clean History:

By using "Squash and Merge," commits of a feature branch are condensed into a single commit on ``master``.

Git guidelines - Conventional Commits

A lightweight specification for formatting commit messages to create a machine-readable history

Commit format

`<type>[optional scope]: <description>`

type: fix, feat, ref, chore, docs, test, merge, rebase

- example: `fix: <...>`

scope: `(<module-name>)` or `!` (breaking change)

- example: `feat(auth)!: <...>`

My Advice

Before implementation think which `type` and `scope` will fit the code you are going to implement. Ideally, each commit contains an isolated, complete change.

fix(infra): resolve permission loops for observability configs



monandszy committed 1 hour ago

chore(infra): add .env.example files for datastores and observability



monandszy committed 2 hours ago

chore(infra): added .env file separation for secrets



monandszy committed 2 hours ago

ref(infra): adjusted mounts to use folders only



monandszy committed 3 hours ago

ref: transitioned 'docker' to 'infra', moved app deployment into 'app' folder



monandszy committed 3 hours ago

Git guidelines - Conventional Commits

- Why?

- **Clear Communication:**

Instantly communicates the nature of changes to teammates without needing to read the actual code.

- **Easier Debugging:**

A structured history allows to quickly scan for recent `feat` or `ref` commits in a specific `scope`.

- **Automation:**

This format enables potential integration with release and documentation tooling.

Git guidelines - Branch Rules

- Enforcing Linear History:

Allowing only squash merging guarantees a "linear history." This means the `master` branch has a clean, chronological history of features and fixes

- Allow only squash merging of pull requests
- Require linear history

- Enforcing the Process:

Human error is inevitable. Branch protection rules physically prevent developers from accidentally pushing unreviewed code directly to `master`.

- Require a pull request before merging
- Require approvals
- Dismiss stale pull request approvals
- Do not allow bypassing the above settings



Code Quality Enhancers

Formatter

An automated tool that enforces a consistent code style across the entire codebase.

Lint

A code analysis tool used to scan source code and flag various programming errors

- **End of Style Debates:**

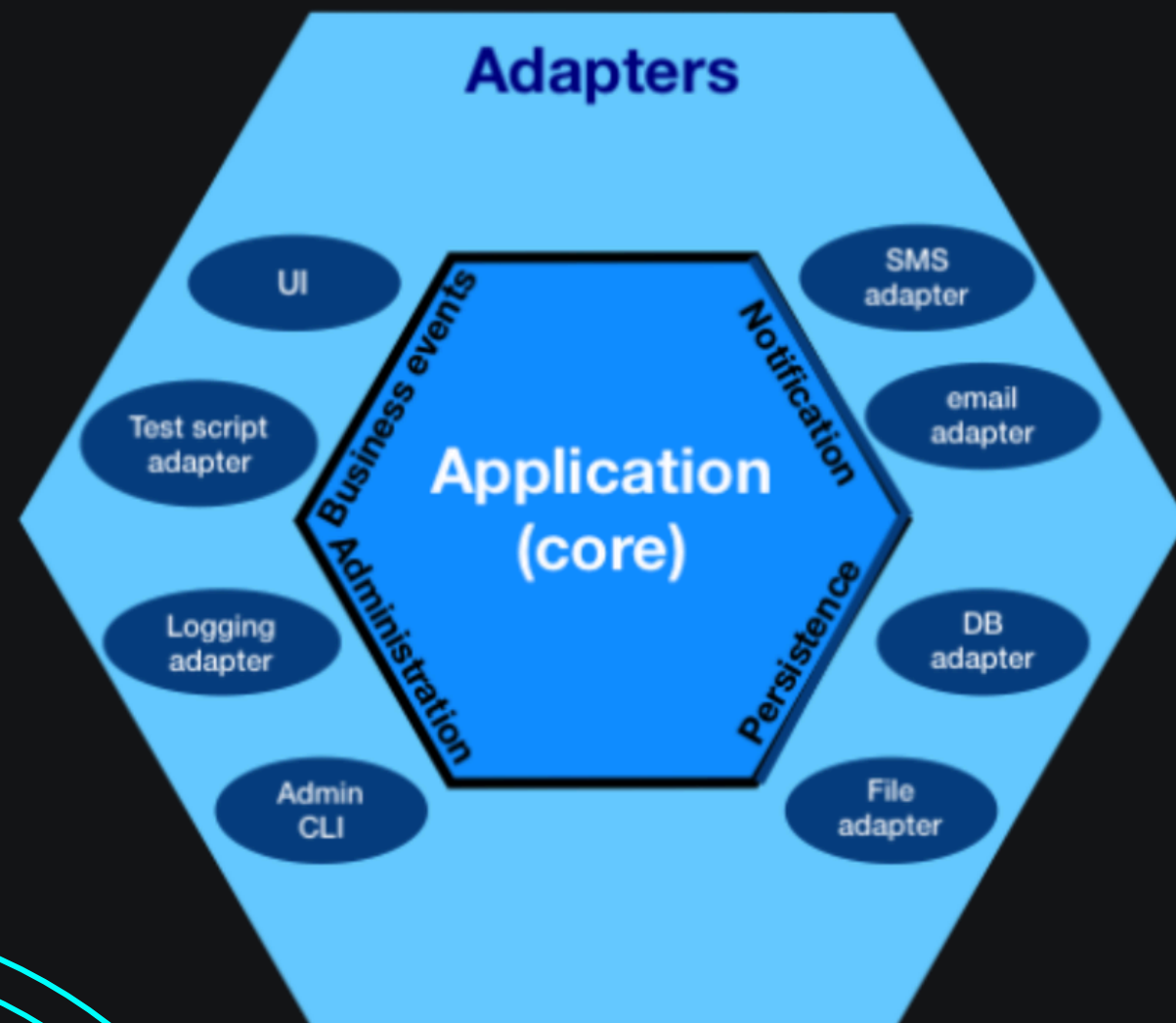
Eliminates subjective nitpicking in code reviews about spacing or formatting.

- **Readability:**

A uniform codebase is significantly easier to navigate.

Hexagonal Architecture (Ports and Adapters)

An architectural pattern that enforces strict separation between the core business logic of an application and the external services



```
src/  
modules/      # Core (The inside of the hexagon)  
  <module-name>/  
    domain/    # Pure classes  
    services/  # Use cases and business logic  
    ports/     # Interfaces  
      in/      # what this module can do  
      out/     # what this module needs (db, api)  
  
entrypoints/  # Inbound Adapters  
  http/       # api endpoints calling modules  
  
infrastructure/ # Outbound Adapters  
  database/    # Implementations of repository ports  
  llm-api/     # Implementations of third-party API calls
```


Hexagonal Architecture - Why ?

- **Isolation of Core Logic:**

Business rules are not polluted with HTTP requests, SQL queries, or framework-specific annotations. The core code remains pure and focused only on solving business problems.

- **Easy Testability:**

External dependencies (Adapters) can be mocked easily in order to write fast, reliable unit tests for the core logic without needing a running database or web server.

- **Technology Agnosticism:**

You can swap out a database engine (e.g., migrating from PostgreSQL to MongoDB) or a web framework without changing a single line of *your* domain logic.

What is a Module?

- **Domain:** The core. Contains pure business entities, value objects, and domain logic. No external dependencies allowed.
- **Services:** Orchestrates business use cases. It fetches data via ports, applies domain logic, and saves the results.
- **Facade (Port):** An interface for the module. Other modules or layers must only interact with this module through its Facade.
- **DAO (IoC):** Data Access Objects. The interfaces (Ports) defined by the core that the infrastructure layer will implement. Uses Inversion of Control to keep dependencies pointing inward.
- **Mappers:** Translate data between the internal Domain models and external representations (like DTOs for an API or Entities for an ORM).

Module (Feature) Development Plan

1. Domain Design

- Identify the core domain classes and their business rules.
- Design the module Use Cases (Facades)
- Create Fixtures for Adapters.

2. Rest API Design

- Define the contract (DTOs, endpoints).
- Set up documentation generation (Swagger/OpenAPI)

3. DevOps Deployment

- Deploy the application, database, and other external dependencies

4. Frontend

- Consume the API

4. Database ERD Design

- Design the persistence layer.
- Set up migration tools and ORM mapping.
- Implement the infrastructure adapters to fulfill the core's Repository ports.

Why?

- **Business Value First:**

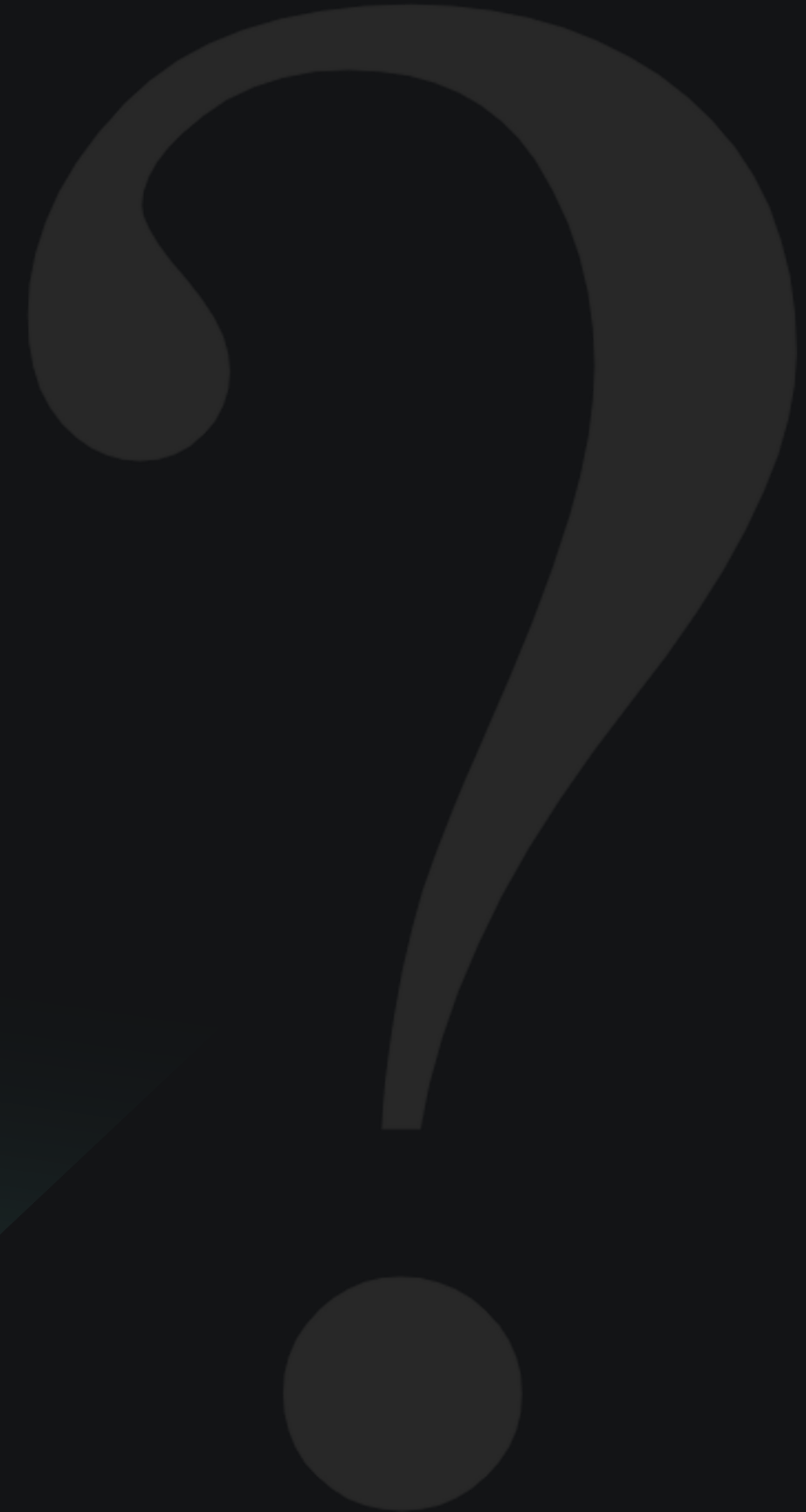
Starting with the Domain Design ensures you are actually solving the business problem before getting bogged down in database schemas or HTTP status codes.

- **Unblocking Teams:**

Defining the REST API contract early allows frontend and backend teams to work strictly in parallel.

- **Deferred Decisions:**

By delaying Database and Infrastructure design, technical choices are delayed until domain logic requirements requirements are fully understood.



Test Strategy

- Facade Tests (Integration)

These are the primary tests, interacting with the application's core logic through its public Facade. External dependencies like databases or external APIs are mocked.

- Unit Tests

Targeted tests for isolated, complex business logic that is difficult to cover exhaustively through Facade tests.

- Entrypoint Tests (API/HTTP)

Verifies the Inbound Adapter layer (e.g., REST API controllers) by mocking the business logic Facade. These tests check routing, request/response serialization, status codes, and middleware.

- Infrastructure Tests (Database/External APIs)

Verifies Outbound Adapters against real external services, using ephemeral instances like Docker containers (Testcontainers) for databases or dedicated sandbox environments for APIs.

Technology Choice

Type-Safe Stack

- Frontend: Next.js + TypeScript + Tailwind
- Backend: Typescript (T3 Stack)

Fullstack

- Kotlin Compose Multiplatform
- Flutter
- Convex

OOP & Architecture Stack

- Frontend: Next.js + TypeScript + Tailwind
- Backend: Java (Spring Boot)

‘Modern’ Stack

- Frontend: Next.js + TypeScript + Tailwind
- Backend: Python (FastAPI)

First Tasks:

Create folder structure

```
frontend/  
  <...>  
backend/  
  <...>  
docs/  
  <...>  
Readme.md
```

Front & Back

- Set up frameworks
- Configure Formatter
- Configure Linter
- Create a Makefile to use those
- Connect an .env file

Backend - first module (User management)

- Or figure out the framework integration.
- Design User domain model
 - Validation Rules
 - Fields, subclasses
- Code the use cases (User)
 - login
 - register
 - Users can update their profile information.
 - Users can upload an avatar (with a default avatar if none provided).
 - Users can add other users as friends and see their online status.
 - Users have a profile page displaying their information.
- Write Integration Tests (For the UseCases mock with fixtures)

Frontend

- Display a single page on the frontend
- Create Component Style
- Design the Login and Register Page
- Design user profile page

API

- Code the API endpoints for the UseCases
- Document the API endpoints
- Write the API web Tests for each endpoint (Mock the UseCases)

Devops & DB

- connect database container
- design user table (v1)
- figure out migrations and .sql versioning
- integrate testcontainers
- create Repositories and test them.